Captive dependencies can cause subtle bugs that are hard to root out, so you should always keep an eye out for them. These captive dependencies are relatively easy to introduce, so always think carefully when registering a singleton service.

WARNING A service should use only dependencies that have a lifetime longer than or equal to the service's lifetime. A service registered as a singleton can safely use only singleton dependencies. A service registered as scoped can safely use scoped or singleton dependencies. A transient service can use dependencies with any lifetime.

At this point, I should mention one glimmer of hope in this cautionary tale: ASP.NET Core automatically checks for these kinds of captive dependencies and throws an exception on application startup if it detects them, or on first use of a captive dependency, as shown in figure 9.11.

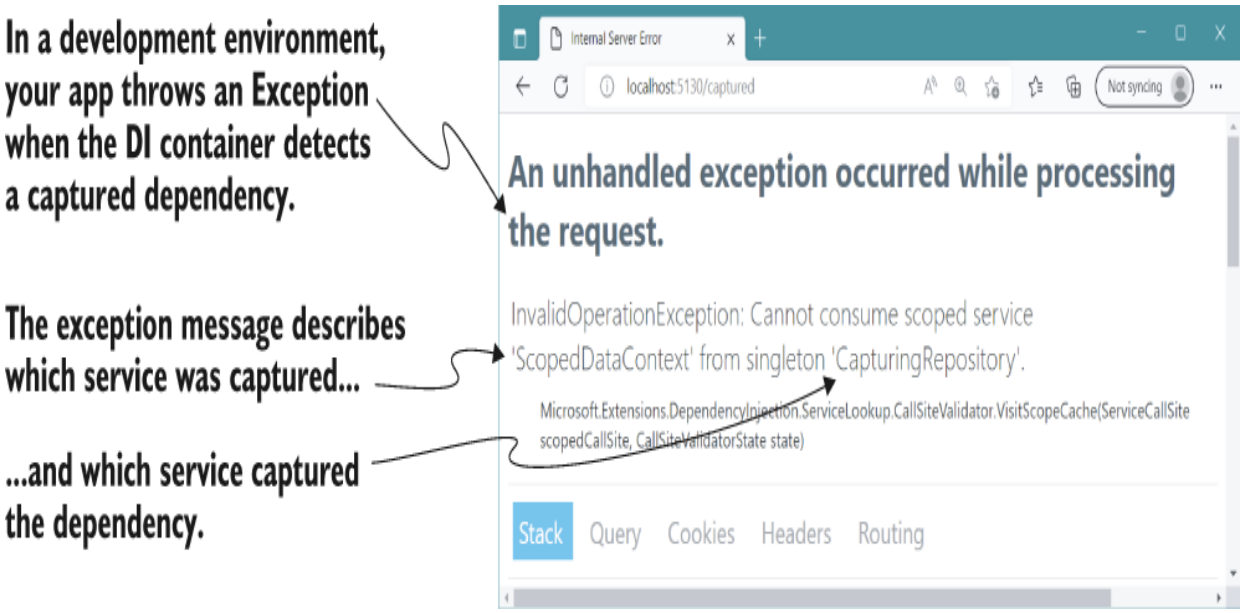


Figure 9.11 When `ValidateScopes` is enabled, the DI container throws an exception when it creates a service with a captive dependency. By default, this check is enabled only for development environments.